Tugas Checkpoint 5

Nama : Labiba Adinda Zahwana

Kelas : C

Bagian 0

Field apa saja yang wajib ada agar automation bisa berjalan?

Field yang wajib ada agar automation SOAR dapat berjalan dengan benar adalah:

Field	**Fungsi**
src_ip	Mengidentifikasi sumber serangan atau attacker
rule_name	Menentukan jenis alert/insiden
event_count	Mengukur jumlah aktivitas mencurigakan
timestamp	Menentukan waktu kejadian
port_count	Digunakan untuk mendeteksi aktivitas port scanning

Tanpa field penting seperti src_ip, rule_name, atau event_count, automation dapat gagal berjalan atau menghasilkan false positive pada proses response.

Bagian 1

Apa risiko jika validasi input diabaikan pada SOAR?

Jika validasi input diabaikan pada SOAR, maka workflow automation dapat memproses payload yang salah, tidak lengkap, atau dimanipulasi sehingga menyebabkan kesalahan response keamanan.

Beberapa risiko yang dapat terjadi:

Risiko	**Dampak**
False Positive	IP normal atau legitimate dapat terblokir
False Negative	Serangan asli tidak terdeteksi
Payload Manipulation	Attacker dapat mengirim alert palsu

Automation Abuse	Workflow dipicu secara tidak sah
Service Disruption	Blocking otomatis dapat mengganggu layanan
Severity Misclassification	Tingkat ancaman salah dihitung
Audit Trail Tidak Valid	Evidence dan logging menjadi tidak akurat

Bagian 2

Mengapa enrichment penting sebelum blocking otomatis?

Enrichment penting karena membantu workflow memahami konteks alert sebelum mengambil keputusan automated response.

Tanpa enrichment:

- IP internal dapat terblokir secara tidak sengaja
- whitelist tidak dikenali
- severity ancaman tidak akurat
- false positive meningkat
- layanan legitimate dapat terganggu

Contohnya, jika IP berasal dari server internal monitoring tetapi tidak dilakukan enrichment, workflow dapat salah menganggap aktivitas tersebut sebagai serangan dan melakukan blocking otomatis.

Dengan enrichment:

- decision making lebih akurat
- false positive berkurang
- blocking lebih aman
- automation lebih reliable
- response dapat disesuaikan dengan tingkat ancaman

Bagian 3

Mengapa blocking sebaiknya bersifat sementara?

Blocking sebaiknya bersifat sementara karena automated response memiliki risiko false positive dan dapat memblokir IP legitimate secara tidak sengaja.

Jika blocking dilakukan permanen:

- user valid dapat kehilangan akses
- layanan legitimate dapat terganggu
- recovery membutuhkan intervensi manual
- risiko operational disruption meningkat

Blocking sementara memungkinkan:

- attacker dihentikan sementara
- sistem tetap aman
- rollback lebih mudah dilakukan
- cooldown automation diterapkan
- false positive dapat diminimalkan

Contohnya, jika IP internal atau user legitimate salah terdeteksi sebagai brute force attacker, maka temporary block akan otomatis berakhir setelah periode tertentu sehingga akses dapat kembali normal tanpa tindakan manual.

Bagian 4

Apa risiko melakukan block tanpa klasifikasi severity?

Melakukan blocking tanpa klasifikasi severity dapat menyebabkan automation memblokir aktivitas yang sebenarnya tidak berbahaya atau legitimate.

Beberapa risiko yang dapat terjadi:

Risiko	**Dampak**
False Positive	Traffic normal ikut terblokir
Gangguan Layanan	User legitimate kehilangan akses
Blocking Berlebihan	Scanning ringan dianggap serangan serius

Operational Disruption	Monitoring atau health check internal gagal
Overreaction Automation	Semua scanning diperlakukan sama
Alert Fatigue	Terlalu banyak block/notifikasi

Contohnya, scanning terhadap port web umum seperti 80 atau 443 belum tentu berbahaya karena dapat berasal dari monitoring tool atau crawler normal. Namun scanning terhadap port sensitif seperti SSH (22) atau RDP (3389) memiliki risiko lebih tinggi sehingga lebih layak untuk diblokir otomatis.

Karena itu klasifikasi severity penting agar response automation tetap proporsional, akurat, dan tidak mengganggu operasional sistem.

Bagian 5

Dampak operasional jika automation tidak memiliki rollback?

Jika automation tidak memiliki rollback atau mekanisme unblock, maka kesalahan response dapat menyebabkan gangguan operasional yang serius.

Beberapa dampak yang dapat terjadi:

Dampak	**Penjelasan**
User Legitimate Terkunci	User valid tidak dapat mengakses layanan
Downtime Operasional	Sistem internal dapat terblokir
Gangguan Monitoring	Monitoring/health check gagal berjalan
Recovery Lambat	Harus dilakukan manual oleh administrator
Risiko False Positive Tinggi	Kesalahan block sulit dipulihkan
Gangguan Layanan Produksi	Service penting dapat tidak dapat diakses

Contohnya, jika IP administrator internal salah terdeteksi sebagai attacker dan diblokir permanen tanpa rollback, maka administrator tidak dapat mengakses server hingga firewall dibuka secara manual.

Karena itu rollback sangat penting agar:

automation tetap aman, recovery lebih cepat, false positive dapat dipulihkan, dan operasional sistem tetap stabil.

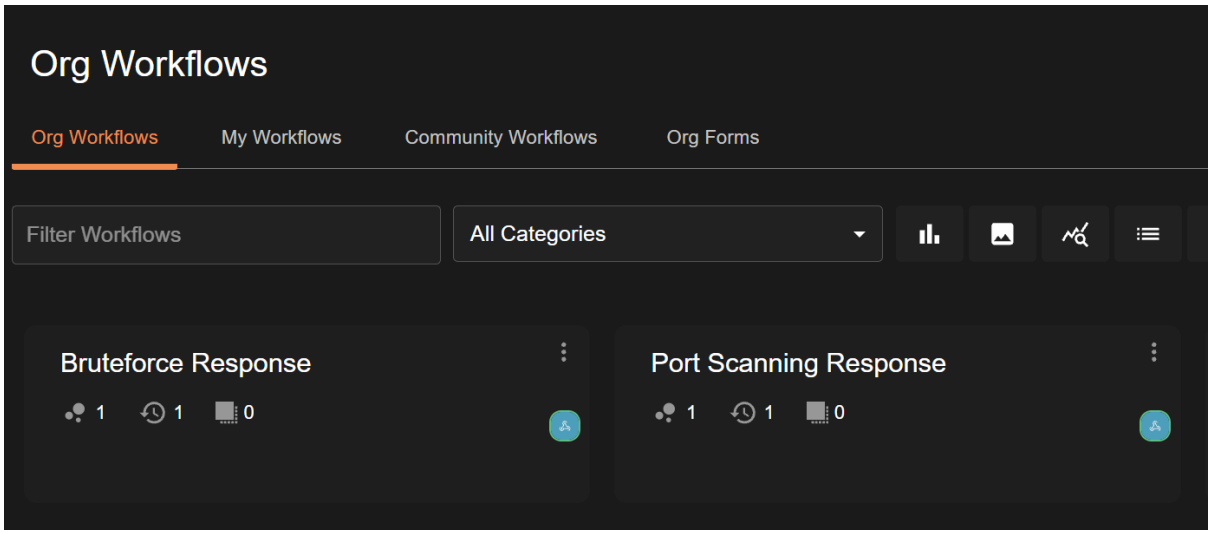
Tugas MINI Project

Diagram alur workflow

```
Webhook Alert
      ↓
Bagian 0 – Webhook Trigger
      ↓
Bagian 1 – Validation & Normalization
      ↓
Bagian 2 – Enrichment & Context Building
      ↓
Attack Classification
   ↙                ↘
Brute Force      Port Scanning
   ↓                  ↓
Block / Notify    Notify / Monitor
   ↓                  ↓
Safety Control & Rollback
```

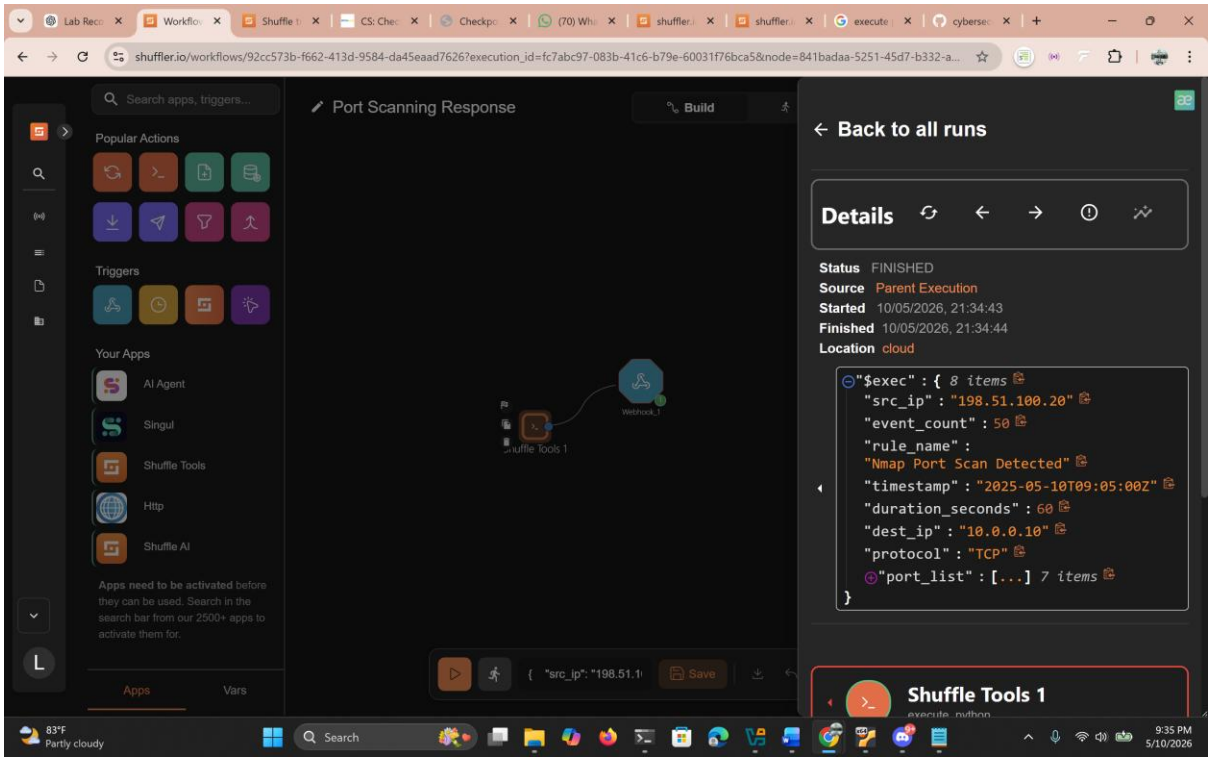
Workflow

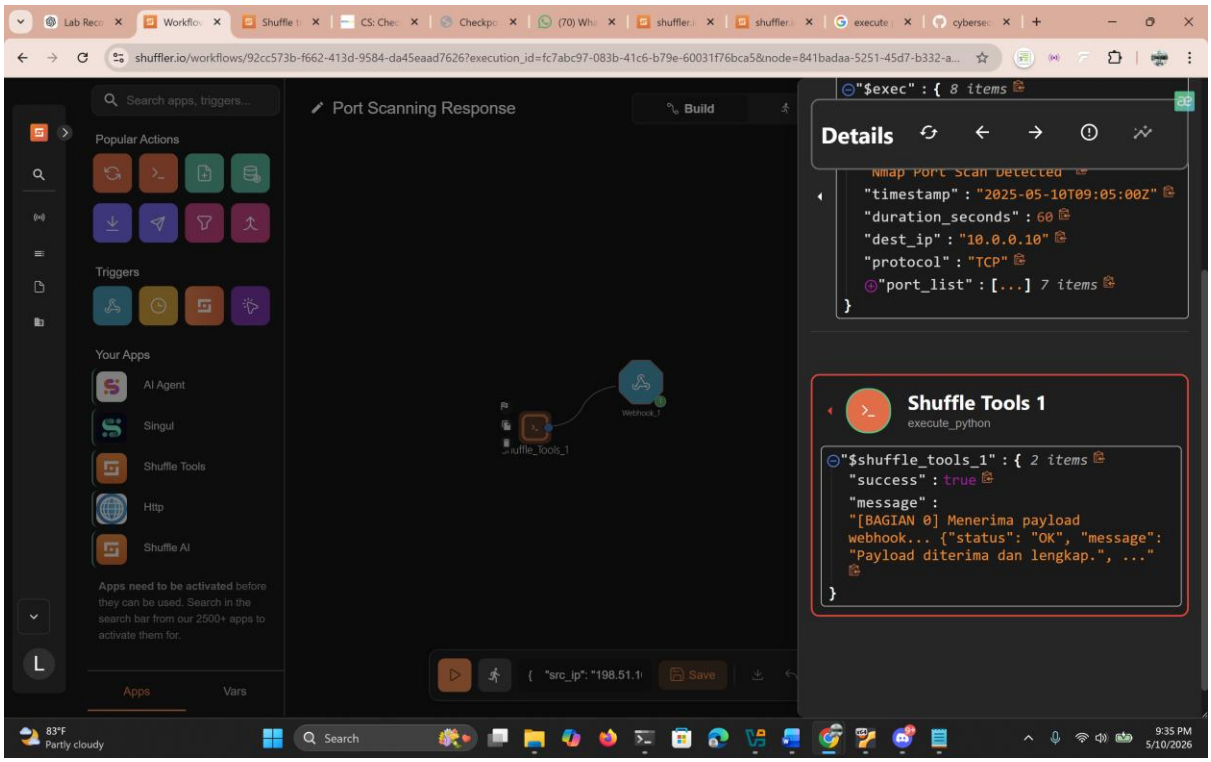
Terdapat dua workflow yang digunakan bruteforce dan port scanning

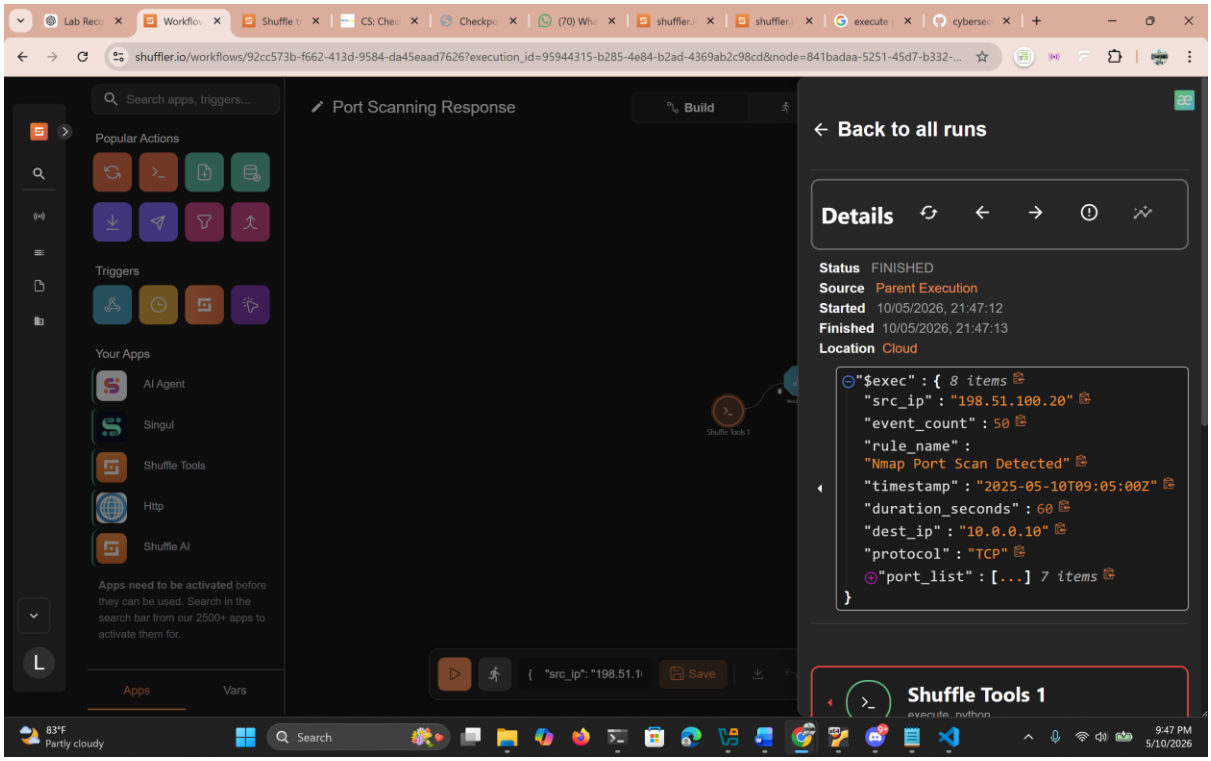


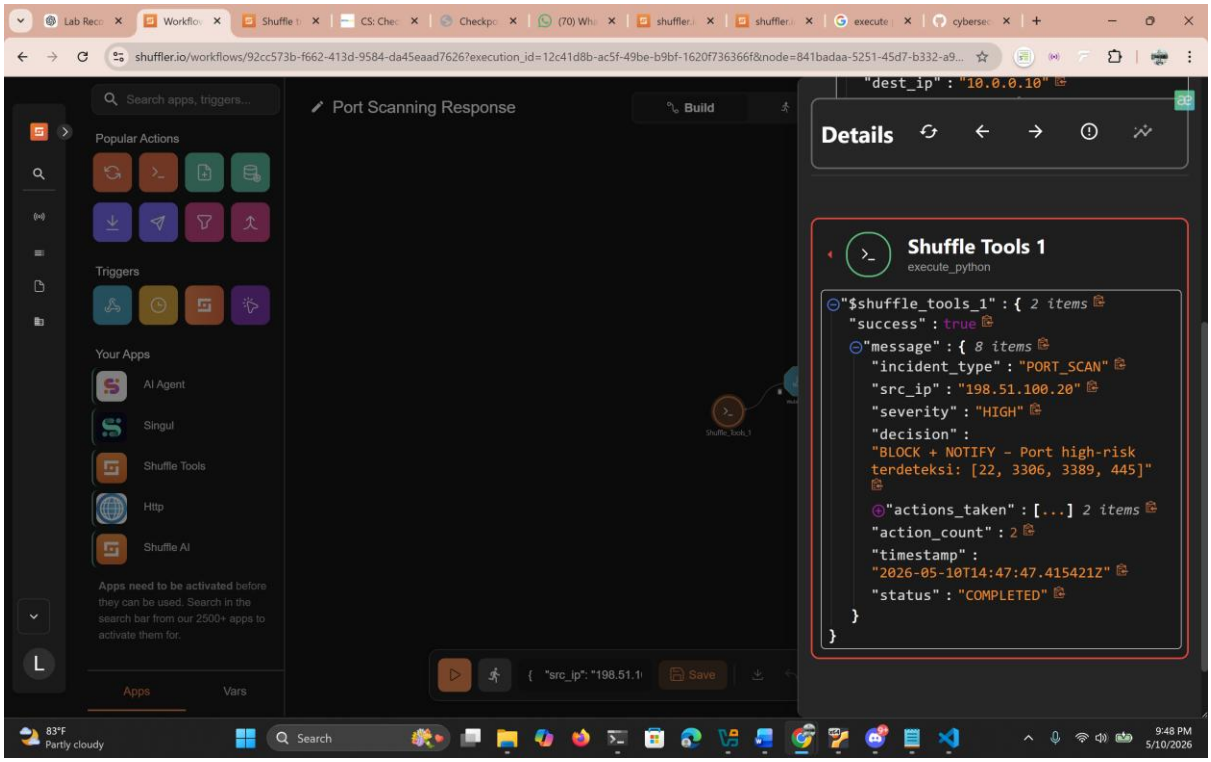
Workflow Port Scanning

```
labibaadinda@labibaadinda:~/Shuffle$ curl -X POST https://shuffler.io/api/v1/hooks/webhook_196ae20d-5b11-4551-ab4b-8c695
2910d06 -H "Content-Type: application/json" -d '{
  "rule_name":"Port Scanning",
  "src_ip":"192.168.0.11",
  "event_count":50,
  "port_count":10,
  "timestamp":"2026-05-10T10:40:00Z"
}'
{"success": true, "execution_id": "5ad56fd1-8ecc-4646-9450-c12518e46b40"}labibaadinda@labibaadinda:~/Shuffle$
```



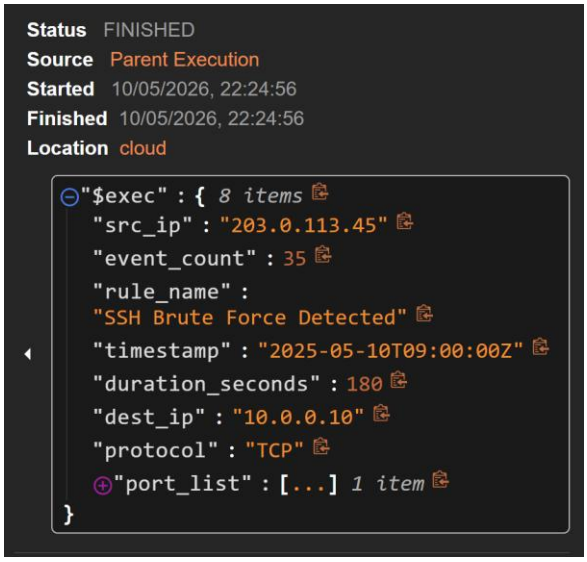


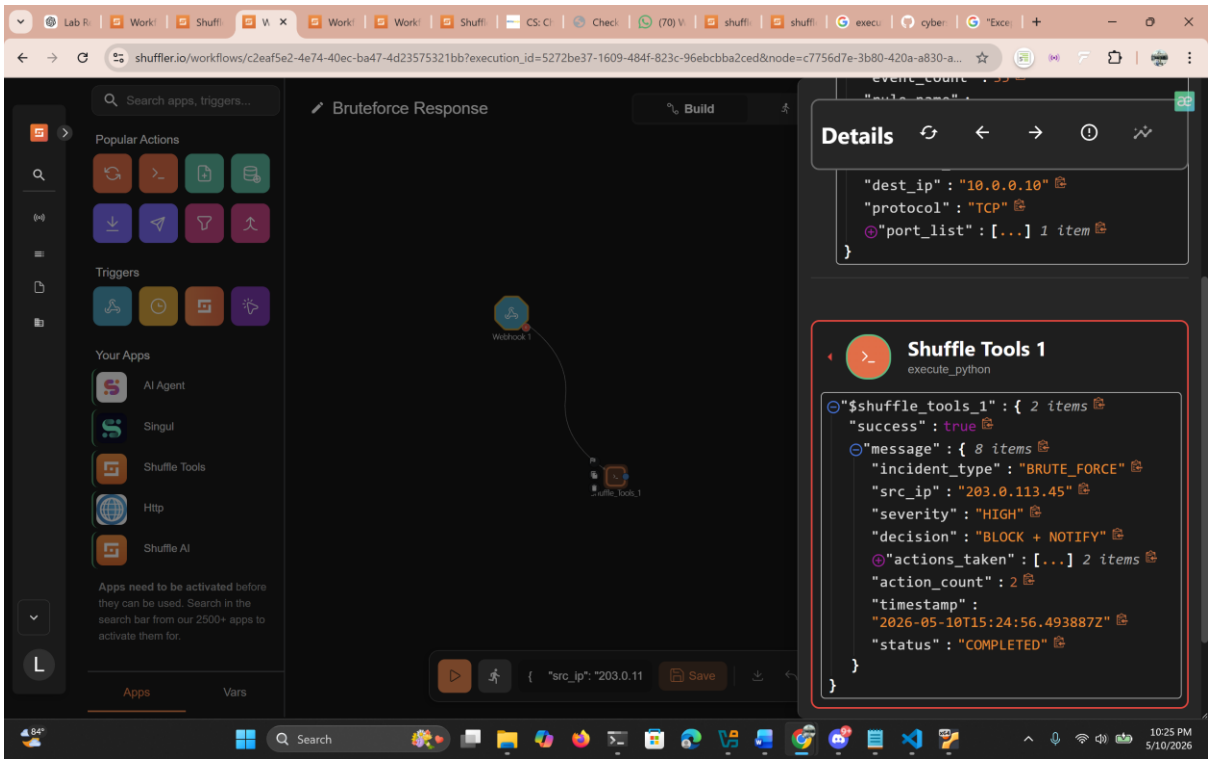

Port Scanning Response
Build
Back to all runs
Details
Status FINISHED
Source Parent Execution
Started 10/05/2026, 21:47:12
Finished 10/05/2026, 21:47:13
Location Cloud
"$exec" : { 8 items
"src_ip" : "198.51.100.20"
"event_count" : 50
"rule_name" : "Nmap Port Scan Detected"
"timestamp" : "2025-05-10T09:05:00Z"
"duration_seconds" : 60
"dest_ip" : "10.0.0.10"
"protocol" : "TCP"
"port_list" : [...] 7 items
Shuffle Tools 1
Search apps, triggers...
Popular Actions
Triggers
Your Apps
AI Agent
Singul
Shuffle Tools
Http
Shuffle AI
Apps need to be activated before they can be used. Search in the search bar from our 2500+ apps to activate them for.
Apps
Vars
Save
9:47 PM
5/10/2026


Port Scanning Response
Build
Details
Shuffle Tools 1
execute_python
"$shuffle_tools_1" : { 2 items
"success" : true
"message" : { 8 items
"incident_type" : "PORT_SCAN"
"src_ip" : "198.51.100.20"
"severity" : "HIGH"
"decision" : "BLOCK + NOTIFY - Port high-risk terdeteksi: [22, 3306, 3389, 445]"
"actions_taken" : [...] 2 items
"action_count" : 2
"timestamp" : "2026-05-10T14:47:47.415421Z"
"status" : "COMPLETED"
Search apps, triggers...
Popular Actions
Triggers
Your Apps
AI Agent
Singul
Shuffle Tools
Http
Shuffle AI
Apps need to be activated before they can be used. Search in the search bar from our 2500+ apps to activate them for.
Apps
Vars
Save
9:48 PM
5/10/2026

Workflow Brute Force

```
labibaadinda@labibaadinda:~$ curl -X POST https://shuffler.io/api/v1/hooks/webhook_196ae20d-5b11-4551-ab4b-8c6952910d06 \
-H "Content-Type: application/json" \
-d '{
    "src_ip": "203.0.113.45",
    "event_count": 35,
    "rule_name": "SSH Brute Force Detected",
    "timestamp": "2025-05-10T09:00:00Z",
    "duration_seconds": 180,
    "dest_ip": "10.0.0.10",
    "protocol": "TCP",
    "port_list": [22]
}'
{
    "success": true,
    "message": ""
```





Python code yang digunakan di Shuffle

```
import json
import re
import subprocess
import datetime
import os

# ──────────────────────────────────────────────────────────────────────
# KONFIGURASI GLOBAL
# ──────────────────────────────────────────────────────────────────────

WHITELIST_IPS = [
    "192.168.1.1",
    "10.0.0.1",
    "127.0.0.1",
    "10.10.10.5",
]

INTERNAL_RANGES = [
    "192.168.",
    "10.",
    "172.16.",
    "172.17.",
    "172.18.",
    "172.19.",
    "172.20.",
    "172.21.",
    "172.22.",
    "172.23.",
    "172.24.",
    "172.25.",
    "172.26.",
    "172.27.",
    "172.28.",
    "172.29.",
    "172.30.",
```

```
    "172.31.",
]

EVIDENCE_LOG_PATH = "/var/log/shuffle_ir_evidence.log"
COOLDOWN_FILE     = "/tmp/shuffle_blocked_ips.json"
BLOCK_DURATION_MINUTES = 30

HIGH_RISK_PORTS = [22, 3389, 445, 3306, 5432, 6379, 27017]
LOW_RISK_PORTS  = [80, 443, 8080, 8443]

def section_0_webhook_trigger(raw_payload: str) -> dict:
    """
    Tujuan : Memastikan Shuffle menerima alert dan payload dapat di-
parse.
    Input  : raw_payload (string JSON dari webhook)
    Output : dict parsed alert atau error

    Di Shuffle, gunakan:
         raw_payload = "{{$exec}}"   ← execution body dari webhook
trigger
    """



    try:
        if isinstance(raw_payload, dict):
            alert = raw_payload
        else:
            alert = json.loads(raw_payload)
    except (json.JSONDecodeError, TypeError) as e:
        result = {
            "status": "ERROR",
           "message": f"Payload tidak valid / bukan JSON: {str(e)}",
            "raw": str(raw_payload)
        }

        return result
```

```
    required_fields = ["src_ip", "event_count", "rule_name"]
    missing = [f for f in required_fields if f not in alert]

    if missing:
        result = {
            "status": "INCOMPLETE",
            "message": f"Field wajib tidak ditemukan: {missing}",
            "alert": alert
        }

        return result

    result = {
        "status": "OK",
        "message": "Payload diterima dan lengkap.",
        "alert": alert
    }

    return result

# ──────────────────────────────────────────────────────────────
# BAGIAN 1 – NORMALISASI & VALIDASI INPUT
# ──────────────────────────────────────────────────────────────

def section_1_normalize_validate(alert: dict) -> dict:
    """
    Tujuan : Mencegah automation berjalan dari data tidak valid /
manipulatif.
    Input  : alert dict (output Bagian 0)
    Output : normalized alert atau error

    Di Shuffle:
        alert = {{$exec.alert}}   ← dari output node Bagian 0
    """



    errors = []
```

```
    # Validasi src_ip
    src_ip = str(alert.get("src_ip", "")).strip()
    ip_pattern = re.compile(
        r'^(\d{1,3}\.){3}\d{1,3}$'
    )
    if not ip_pattern.match(src_ip):
        errors.append(f"src_ip tidak valid: '{src_ip}'")
    else:
        parts = src_ip.split(".")
        if any(int(p) > 255 for p in parts):
            errors.append(f"src_ip oktet out of range: '{src_ip}'")

    # Validasi event_count
    try:
        event_count = int(alert.get("event_count", -1))
        if event_count < 0:
            errors.append("event_count harus >= 0")
    except (ValueError, TypeError):
        errors.append("event_count bukan integer")
        event_count = 0

    # Validasi rule_name
    rule_name = str(alert.get("rule_name", "")).strip()
    if not rule_name:
        errors.append("rule_name kosong")

    # Normalisasi timestamp
                timestamp        =        alert.get("timestamp",
datetime.datetime.utcnow().isoformat() + "Z")

    # Normalisasi opsional: port_list
    port_list = alert.get("port_list", [])
    if isinstance(port_list, str):
        try:
            port_list = json.loads(port_list)
        except Exception:
```

```
            port_list = []

    if errors:
        result = {
            "status": "INVALID",
            "errors": errors,
            "original_alert": alert
        }

        return result

    normalized = {
        "src_ip": src_ip,
        "event_count": event_count,
        "rule_name": rule_name,
        "timestamp": timestamp,
        "port_list": port_list,
        "duration_seconds": alert.get("duration_seconds", 300),
        "dest_ip": alert.get("dest_ip", ""),
        "protocol": alert.get("protocol", "").upper(),
    }

    result = {
        "status": "VALID",
        "normalized_alert": normalized
    }

    return result

# ──────────────────────────────────────────────────────────────
# BAGIAN 2 – ENRICHMENT & CONTEXT BUILDING
# ──────────────────────────────────────────────────────────────

def section_2_enrichment(normalized_alert: dict) -> dict:
    """
    Tujuan : Menambahkan konteks sebelum keputusan response diambil.
    Input  : normalized_alert (output Bagian 1)
```

```
    Output : enriched context

    Di Shuffle:
        normalized_alert = {{$exec.normalized_alert}}
    """



    src_ip = normalized_alert["src_ip"]
    event_count = normalized_alert["event_count"]
    rule_name = normalized_alert["rule_name"].lower()
    port_list = normalized_alert.get("port_list", [])

    # Cek apakah IP internal
      is_internal  =  any(src_ip.startswith(prefix)  for  prefix  in
INTERNAL_RANGES)

    # Cek whitelist
    is_whitelisted = src_ip in WHITELIST_IPS

    # Severity hint berdasarkan rule + event count
    if event_count >= 50:
        severity_hint = "CRITICAL"
    elif event_count >= 20:
        severity_hint = "HIGH"
    elif event_count >= 10:
        severity_hint = "MEDIUM"
    else:
        severity_hint = "LOW"

    # Override severity untuk whitelist / internal
    if is_whitelisted or is_internal:
        severity_hint = "INFO"

    # Deteksi tipe serangan berdasarkan rule_name
    if any(k in rule_name for k in ["brute", "ssh", "login", "auth",
"fail"]):
        attack_type = "BRUTE_FORCE"
```

```
    elif any(k in rule_name for k in ["scan", "recon", "probe",
"sweep", "nmap"]):
        attack_type = "PORT_SCAN"
    else:
        attack_type = "UNKNOWN"

    # Analisis port risk (untuk scanning)
    high_risk_detected = [p for p in port_list if int(p) in
HIGH_RISK_PORTS]
    low_risk_only = all(int(p) in LOW_RISK_PORTS for p in port_list)
if port_list else False

    context = {
        "src_ip": src_ip,
        "is_internal": is_internal,
        "is_whitelisted": is_whitelisted,
        "severity_hint": severity_hint,
        "attack_type": attack_type,
        "high_risk_ports_detected": high_risk_detected,
        "low_risk_only": low_risk_only,
        "event_count": event_count,
        "port_count": len(port_list),
        "enrichment_time": datetime.datetime.utcnow().isoformat() +
"Z",
        "normalized_alert": normalized_alert
    }


    return context

# ──────────────────────────────────────────────────────────────
# BAGIAN 3 – PLAYBOOK RESPONSE BRUTE FORCE
# ──────────────────────────────────────────────────────────────

def section_3_bruteforce_response(context: dict, use_ufw: bool =
False) -> dict:
    """
```

```
    Tujuan : Otomasi response untuk brute force login.
    Logic  : event_count >= 20 dalam <= 5 menit AND bukan whitelist.
     Input  : context (output Bagian 2), use_ufw (True = eksekusi
UFW)
    Output : response report

    Di Shuffle:
        context  = {{$exec}}           ← full context dari Bagian 2
        use_ufw  = False               ← set True jika ingin blokir
nyata
    """



    src_ip       = context["src_ip"]
    event_count  = context["event_count"]
    is_whitelisted = context["is_whitelisted"]
    severity     = context["severity_hint"]
                            duration_s                                =
context["normalized_alert"].get("duration_seconds", 300)
    rule_name    = context["normalized_alert"]["rule_name"]

    actions_taken = []
    decision = ""

    # ── Safety gate ──────────────────────────────────────────
    if is_whitelisted:
        decision = "SKIP – IP ada di whitelist"
           result  =  _build_response_report("BRUTE_FORCE",  src_ip,
decision, actions_taken, severity)

        return result

    if context["is_internal"]:
        decision = "NOTIFY_ONLY – IP internal, tidak diblokir"
         actions_taken.append(_notify(src_ip, rule_name, severity,
note="Internal IP, blokir dilewati"))
```

```
            result = _build_response_report("BRUTE_FORCE", src_ip,
decision, actions_taken, severity)

        return result

    # ── Decision logic ──────────────────────────────────────
    # event_count >= 20 dalam <= 5 menit (300 detik)
    if event_count >= 20 and duration_s <= 300:
        decision = "BLOCK + NOTIFY"

        # Cooldown check
        if _is_in_cooldown(src_ip):
            decision = "SKIP - Sudah dalam cooldown, tidak duplikat
blokir"
             result = _build_response_report("BRUTE_FORCE", src_ip,
decision, actions_taken, severity)

            return result

        # Blokir sementara
             block_result = _block_ip(src_ip, use_ufw=use_ufw,
reason="BruteForce")
        actions_taken.append(block_result)

        # Catat cooldown
        _add_to_cooldown(src_ip)

        # Notifikasi
        actions_taken.append(_notify(src_ip, rule_name, severity))

    elif event_count >= 10:
        decision = "NOTIFY_ONLY - event_count sedang, monitoring"
         actions_taken.append(_notify(src_ip, rule_name, severity,
note="Threshold rendah, hanya notify"))

    else:
        decision = "LOG_ONLY - event_count rendah"
```

```
    result = _build_response_report("BRUTE_FORCE", src_ip, decision,
actions_taken, severity)

    return result

# ──────────────────────────────────────────────────────────────────
# BAGIAN 4 – PLAYBOOK RESPONSE PORT SCANNING
# ──────────────────────────────────────────────────────────────────

def section_4_portscan_response(context: dict, use_ufw: bool =
False) -> dict:
    """
    Tujuan : Menangani aktivitas reconnaissance / port scanning.
    Logic  : port_count >= 5 ATAU event_count tinggi.
    Input  : context (output Bagian 2)
    Output : response report

    Di Shuffle:
        context = {{$exec}}
        use_ufw = False
    """


    src_ip              = context["src_ip"]
    event_count         = context["event_count"]
    port_count          = context["port_count"]
    is_whitelisted      = context["is_whitelisted"]
    severity            = context["severity_hint"]
    high_risk_ports     = context["high_risk_ports_detected"]
    low_risk_only       = context["low_risk_only"]
    rule_name           = context["normalized_alert"]["rule_name"]

    actions_taken = []
    decision = ""

    # ── Safety gate ──────────────────────────────────────────
```

```
    if is_whitelisted or context["is_internal"]:
        decision = "SKIP - Whitelisted / IP internal"
            result  =  _build_response_report("PORT_SCAN",  src_ip,
decision, actions_taken, severity)

        return result

    # ── Severity classification ──────────────────────────────
    triggered = (port_count >= 5) or (event_count >= 30)

    if triggered:
        if high_risk_ports:
            # High-risk: SSH, RDP, DB ports terdeteksi → BLOCK
            decision = f"BLOCK + NOTIFY - Port high-risk terdeteksi:
{high_risk_ports}"
            severity = "HIGH"

            if not _is_in_cooldown(src_ip):
                 block_result = _block_ip(src_ip, use_ufw=use_ufw,
reason="PortScan-HighRisk")
                actions_taken.append(block_result)
                _add_to_cooldown(src_ip)
            else:
                decision += " (sudah dalam cooldown)"

                  actions_taken.append(_notify(src_ip,  rule_name,
severity, note=f"Port kritis: {high_risk_ports}"))

        elif low_risk_only:
            # Low-risk: hanya HTTP/HTTPS → hanya notify
            decision = "NOTIFY_ONLY - Hanya port low-risk (80/443)"
            severity = "LOW"
                  actions_taken.append(_notify(src_ip,  rule_name,
severity, note="Scan ke port web saja"))

        else:
            # Mixed ports → notify + monitor
```

```
            decision = "NOTIFY + MONITOR – Port campuran"
            severity = "MEDIUM"
                  actions_taken.append(_notify(src_ip,  rule_name,
severity, note="Port campuran, perlu monitoring"))

    else:
        decision = "LOG_ONLY – Ambang batas scanning belum tercapai"

    result = _build_response_report("PORT_SCAN", src_ip, decision,
actions_taken, severity)

    return result

# ──────────────────────────────────────────────────────────────────
# BAGIAN 5 – SAFETY CONTROLS (Whitelist, Cooldown, Rollback)
# ──────────────────────────────────────────────────────────────────

def section_5_safety_controls(action: str, target_ip: str, use_ufw:
bool = False) -> dict:
    """
     Tujuan : Manajemen keamanan automation (whitelist, cooldown,
rollback/unblock).
    Input  :
        action    = "add_whitelist" | "remove_whitelist" |
                    "unblock" | "check_cooldown" | "clear_cooldown"
        target_ip = IP yang ditarget
        use_ufw   = True jika ingin unblock nyata via UFW

    Di Shuffle, buat node dengan parameter:
        action    = "{{$exec.action}}"
        target_ip = "{{$exec.target_ip}}"

    Contoh payload webhook untuk trigger rollback:
        {"action": "unblock", "target_ip": "1.2.3.4"}
    """
```

```
    result = {"action": action, "target_ip": target_ip, "status": ""}

    if action == "add_whitelist":
        if target_ip not in WHITELIST_IPS:
            WHITELIST_IPS.append(target_ip)
           result["status"] = f"{target_ip} ditambahkan ke whitelist (runtime)"
        else:
            result["status"] = f"{target_ip} sudah ada di whitelist"

    elif action == "remove_whitelist":
        if target_ip in WHITELIST_IPS:
            WHITELIST_IPS.remove(target_ip)
           result["status"] = f"{target_ip} dihapus dari whitelist"
        else:
            result["status"] = f"{target_ip} tidak ada di whitelist"

    elif action == "unblock":
        unblock_result = _unblock_ip(target_ip, use_ufw=use_ufw)
        _remove_from_cooldown(target_ip)
        result["status"] = unblock_result
        result["rollback"] = True


    elif action == "clear_cooldown":
        _remove_from_cooldown(target_ip)
        result["status"] = f"Cooldown untuk {target_ip} dihapus"

    else:
        result["status"] = f"Action tidak dikenal: {action}"



    return result

# ──────────────────────────────────────────────────────────────────
```

```
# HELPER FUNCTIONS (dipakai di semua bagian)
# ──────────────────────────────────────────────────────────────

def _block_ip(ip: str, use_ufw: bool = False, reason: str = "") ->
dict:
    """Blokir IP via UFW atau dummy log."""
    timestamp = datetime.datetime.utcnow().isoformat() + "Z"

    if use_ufw:
        try:
            cmd = ["sudo", "ufw", "deny", "from", ip, "to", "any"]
              subprocess.run(cmd, check=True, capture_output=True,
text=True)
            status = f"UFW: IP {ip} diblokir ({reason})"
        except subprocess.CalledProcessError as e:
            status = f"UFW ERROR: {e.stderr}"
    else:
        # Dummy mode – cocok untuk testing di Shuffle tanpa sudo
        status = f"[DUMMY] Block IP {ip} – alasan: {reason}"

    action = {
        "action": "BLOCK",
        "ip": ip,
        "reason": reason,
        "mode": "ufw" if use_ufw else "dummy",
        "status": status,
        "timestamp": timestamp,
        "block_until": (
            datetime.datetime.utcnow() +
            datetime.timedelta(minutes=BLOCK_DURATION_MINUTES)
        ).isoformat() + "Z"
    }

    return action

def _unblock_ip(ip: str, use_ufw: bool = False) -> str:
    """Rollback: hapus blokir IP."""
```

```
    if use_ufw:
        try:
              cmd = ["sudo", "ufw", "delete", "deny", "from", ip,
"to", "any"]
              subprocess.run(cmd, check=True, capture_output=True,
text=True)
            return f"UFW: Blokir {ip} dihapus (rollback)"
        except subprocess.CalledProcessError as e:
            return f"UFW unblock ERROR: {e.stderr}"
    else:
        return f"[DUMMY] Unblock IP {ip} berhasil"

def _notify(ip: str, rule: str, severity: str, note: str = "") ->
dict:
    """Simulasi notifikasi (print / bisa diteruskan ke Shuffle HTTP
node)."""
    msg = {
        "action": "NOTIFY",
        "channel": "shuffle_notification",
        "severity": severity,
        "ip": ip,
        "rule": rule,
        "note": note,
        "timestamp": datetime.datetime.utcnow().isoformat() + "Z",
       "message": f"[{severity}] Insiden dari {ip} - {rule}. {note}"
    }

    return msg

def _is_in_cooldown(ip: str) -> bool:
    """Cek apakah IP masih dalam periode cooldown."""
    try:
        if not os.path.exists(COOLDOWN_FILE):
            return False
        with open(COOLDOWN_FILE, "r") as f:
            data = json.load(f)
        entry = data.get(ip)
```

```
        if not entry:
            return False
                                                blocked_at          =
datetime.datetime.fromisoformat(entry["blocked_at"].replace("Z",
""))
                    elapsed    =    (datetime.datetime.utcnow()    -
blocked_at).total_seconds() / 60
        return elapsed < BLOCK_DURATION_MINUTES
    except Exception:
        return False

def _add_to_cooldown(ip: str):
    """Tambahkan IP ke cooldown tracker."""
    try:
        data = {}
        if os.path.exists(COOLDOWN_FILE):
            with open(COOLDOWN_FILE, "r") as f:
                data = json.load(f)
                                data[ip]       =       {"blocked_at":
datetime.datetime.utcnow().isoformat() + "Z"}
        with open(COOLDOWN_FILE, "w") as f:
            json.dump(data, f, indent=2)
    except Exception as e:
        print(e)


def _remove_from_cooldown(ip: str):
    """Hapus IP dari cooldown tracker."""
    try:
        if not os.path.exists(COOLDOWN_FILE):
            return
        with open(COOLDOWN_FILE, "r") as f:
            data = json.load(f)
        data.pop(ip, None)
        with open(COOLDOWN_FILE, "w") as f:
            json.dump(data, f, indent=2)
    except Exception as e:
```

```
        print(e)


def     _build_response_report(incident_type:     str,     src_ip:
str,decision: str, actions: list, severity: str) -> dict:
    """Bangun response report standar."""
    return {
        "incident_type": incident_type,
        "src_ip": src_ip,
        "severity": severity,
        "decision": decision,
        "actions_taken": actions,
        "action_count": len(actions),
        "timestamp": datetime.datetime.utcnow().isoformat() + "Z",
        "status": "COMPLETED"
    }

# ──────────────────────────────────────────────────────────────
# CONTOH PAYLOAD SIMULASI (untuk testing lokal / Shuffle webhook)
# ──────────────────────────────────────────────────────────────

SAMPLE_BRUTE_FORCE_PAYLOAD = {
    "src_ip": "203.0.113.45",
    "event_count": 35,
    "rule_name": "SSH Brute Force Detected",
    "timestamp": "2025-05-10T09:00:00Z",
    "duration_seconds": 180,
    "dest_ip": "10.0.0.10",
    "protocol": "TCP",
    "port_list": [22]
}

SAMPLE_PORT_SCAN_PAYLOAD = {
    "src_ip": "198.51.100.20",
    "event_count": 50,
    "rule_name": "Nmap Port Scan Detected",
    "timestamp": "2025-05-10T09:05:00Z",
```

```
    "duration_seconds": 60,
    "dest_ip": "10.0.0.10",
    "protocol": "TCP",
    "port_list": [22, 80, 443, 3306, 3389, 8080, 445]
}

raw_payload = """$exec"""
# Bagian 0
payload = json.loads(raw_payload)
b0 = section_0_webhook_trigger(payload)
if b0["status"] != "OK":
    print("Workflow dihentikan di Bagian 0.")
else:
    # Bagian 1
    b1 = section_1_normalize_validate(b0["alert"])
    if b1["status"] != "VALID":
        print("Workflow dihentikan di Bagian 1.")
    else:
        # Bagian 2
        b2 = section_2_enrichment(b1["normalized_alert"])
        # Bagian 3
        if "Brute Force" in payload["rule_name"]:
            b3 = section_3_bruteforce_response(b2, use_ufw=False)
            print(json.dumps(b3, indent=2))
        else:
            b4  = section_4_portscan_response(b2, use_ufw=False)
            print(json.dumps(b4, indent=2))
```